Preparación antes de grabar (empezar YA · ~4 horas antes)

Reparto: el **Integrante 2** es el **dueño del repositorio de GitHub** y hace el pipeline y la prueba de carga. El **Integrante 1** hace la arquitectura, el servicio y Terraform. (Hay que hacerlo así porque en un repositorio personal solo el dueño puede ver y editar los *Secrets*.)

Todo se escribe en Git Bash. En VS Code: **Terminal** → **New Terminal**, luego la flecha ˅ junto al **+** → **Git Bash**.

Cuidado con el Learner Lab: la sesión dura **4 horas**. Antes de grabar, hagan clic otra vez en **Start Lab**: el contador vuelve a 4 horas. Si las credenciales cambian, cópienlas de nuevo (paso 2).

⏱ R − 4:00 · Paso 1. Subir el código a GitHub (solo el Integrante 2)

1. Entrar a https://github.com → botón verde **New** (o **+** → **New repository**).
2. Nombre: `dgiti-ef` · **Public** · **no** marcar "Add a README" · **Create repository**.
3. En Git Bash, en la carpeta del proyecto (`dgiti-ef`): `bash git remote add origin https://github.com/SU_USUARIO/dgiti-ef.git git push -u origin main` Si pide iniciar sesión, se abre el navegador: autorizar.
4. En GitHub: **Settings** → **Collaborators** → **Add people** → escribir el usuario del Integrante 1 → **Add**. El Integrante 1 acepta la invitación que le llega por correo.

⏱ R − 3:50 · Paso 1b. Clonar (solo el Integrante 1)

```
cd ~/Desktop
git clone https://github.com/USUARIO_DEL_INTEGRANTE_2/dgiti-ef.git
cd dgiti-ef
```

⏱ R − 3:45 · Paso 2. Credenciales de AWS Academy (LOS DOS, cada uno en su PC)

1. https://awsacademy.instructure.com → **Cursos** → el curso del Learner Lab → **Módulos** → **Launch AWS Academy Learner Lab**.
2. Clic en ▶ **Start Lab** y esperar a que el círculo **AWS** se ponga **verde** (1 a 3 minutos).
3. Clic en ⓘ **AWS Details** → junto a *AWS CLI* clic en **Show** → copiar todo el bloque `[default] … aws_session_token=…` .
4. En Git Bash: `bash mkdir -p ~/.aws && code ~/.aws/credentials` Se abre el archivo en VS Code: **borrar todo**, **pegar** el bloque y **guardar** (Ctrl+S).
5. Comprobar: `bash aws sts get-caller-identity` Debe mostrar su `Account` y un `Arn` con `voclabs` . Si dice `ExpiredToken` , repetir 3 y 4.

⏱ R − 3:40 · Paso 3. Desplegar con Terraform (LOS DOS, en su cuenta)

```
cd terraform
cp terraform.tfvars.example terraform.tfvars
code terraform.tfvars
```

Cambiar `db_password` por una clave propia, **solo letras y números**, de al menos 8 caracteres, por ejemplo `Dgiti2026Clave`. Guardar.

```
terraform init
terraform plan
terraform apply
```

Escribir `yes`. **Tarda unos 20 a 25 minutos**, sobre todo RDS y su réplica. No cierren la terminal. Al terminar, ver los resultados:

```
terraform output
cd ..
```

> Si sale un error de AWS (por ejemplo `AccessDenied` o `not authorized`), copien el mensaje completo y pásenlo para corregirlo. Vuelvan a ejecutar `terraform apply`: retoma desde donde quedó.

⏱ R − 3:10 · Paso 4. Comprobar que todo funciona (LOS DOS)

Esperar unos 3 minutos después del apply para que arranquen las EC2, y luego:

```
./scripts/demo_servicio.sh
```

Tiene que salir: - `/health` → `"status": "ok"` - `/status` → `"ok": true`, con `primary` y `replica` en `"ok": true` - `GET /api/visits` → `"read_from": "replica"`

Además: - Abrir en el navegador la URL `app_url` y recargar: debe cambiar la instancia o la zona. - Consola → **EC2** → **Target Groups** → **dgiti-ef-tg** → **Targets**: 2 instancias **healthy**.

Si `/status` da 503 en la réplica, esperen 5 minutos: la réplica puede tardar más en quedar disponible.

⏱ R − 3:00 · Paso 5. Configurar y correr el pipeline (solo el Integrante 2)

1. GitHub → repo → **Settings** → **Secrets and variables** → **Actions**.
2. Pestaña **Secrets** → **New repository secret**, tres veces, con los valores del bloque de AWS Details (paso 2): | Name | Valor | |---|---| | `AWS_ACCESS_KEY_ID` | el valor de `aws_access_key_id` | | `AWS_SECRET_ACCESS_KEY` | el valor de `aws_secret_access_key` | | `AWS_SESSION_TOKEN` | el valor de `aws_session_token` (es muy largo; cópienlo entero) |
3. Pestaña **Variables** → **New repository variable** → Name `AWS_DEPLOY_ENABLED`, Value `true`.
4. **Actions** → **CI-CD** → **Run workflow** → **Run workflow**.

5. Esperar unos **10 minutos** a que las **3 etapas** queden en **verde ✓**. Esa es la ejecución que van a mostrar en el video.

> Si la etapa 3 falla con `ExpiredToken`, las credenciales cambiaron: actualicen los 3 secretos y vuelvan a correrlo.

⏱ R − 2:30 · Paso 6. Diagrama en PNG (cualquiera de los dos)

1. Abrir https://app.diagrams.net → **Device** (o **Open Existing Diagram**) → elegir `docs/arquitectura-aws.drawio`.
2. **File** → **Export as** → **PNG...** → **Zoom 200 %** → **Export** → guardarlo como `docs/arquitectura-aws.png`.
3. Subirlo (Integrante 2): `bash git add docs/arquitectura-aws.png && git commit -m "Diagrama en PNG" && git push` Ese push vuelve a correr el pipeline: déjenlo terminar en verde (10 min) **antes** de la prueba previa.

⏱ R − 2:00 · Paso 7. Prueba de carga previa (solo el Integrante 2)

Sirve para que CloudWatch ya muestre el escalado aunque en vivo no alcance a verse:

```
./scripts/prueba_carga.sh larga
```

Dura unos **7 minutos**. Al terminar, revisar **CloudWatch** → **Dashboards** → **dgiti-ef-dashboard** (rango 1h): deben verse los picos y las **instancias en servicio** subiendo a 3 o 4. **No cierren esa terminal** (terminal 3): es el respaldo si k6 no termina a tiempo en el video.

⏱ R − 1:45 · Paso 8. Ensayo completo cronometrado (LOS DOS)

- Sigan `docs/GUION_CORTO.md` con cronómetro: **debe durar menos de 6 minutos**.
- **El Integrante 1**, en el ensayo, usa `app_version=1.1.0`; **en la grabación real usa** `1.2.0`, para que el plan vuelva a mostrar un cambio.
- **El Integrante 2**: después del ensayo, esperen unos **15 minutos** a que el ASG vuelva a 2 instancias.

⏱ R − 0:15 · Paso 9. Justo antes de grabar

Los dos: - [ ] Learner Lab → **Start Lab** otra vez (renueva las 4 horas) → `aws sts get-caller-identity` funciona. - [ ] Letra de la terminal grande (**Ctrl +**), notificaciones apagadas y otras ventanas cerradas.

Pestañas abiertas del Integrante 1: - [ ] Learner Lab - [ ] Diagrama (PNG o diagrams.net) - [ ] Consola → VPC → Subnets - [ ] Navegador con `app_url` - [ ] VS Code con la carpeta `terraform/` y Git Bash en la **raíz del repo**

Pestañas abiertas del Integrante 2: - [ ] Learner Lab - [ ] GitHub → `ci-cd.yml` - [ ] GitHub → Settings → Secrets - [ ] GitHub → Actions → la ejecución verde - [ ] S3 → `dgiti-ef-artifacts-…` → `ci/` - [ ] EC2 → Target Groups → dgiti-ef-tg - [ ] CloudWatch → dgiti-ef-dashboard (rango **3h**) - [ ] EC2 → Auto Scaling

Groups → dgiti-ef-asg → Activity - [ ] Git Bash **terminal 1** (k6), **terminal 2** (check_balancer) y **terminal 3** con el resultado de la prueba previa, todas en la raíz del repo - [ ] Comprobar en Auto Scaling Groups que **Desired capacity = 2**

Después de grabar (LOS DOS)

```
terraform -chdir=terraform destroy
```

Escribir `yes`. Así no se gasta el presupuesto del lab. Tarda unos 10 a 15 minutos.